

Malware Analysis Report

ATM Jackpotting Sample: Ploutus Variant (.NET)

SHA-256: fe1634318e27e3af856506d49a54d1d12e1cf650cbc31eeb0c805949edc8fc85

Prepared by: Abdullah Zia

1. Introduction

This report documents the complete static and dynamic analysis of a suspected ATM jackpotting malware sample stored as Sample 2 in the analysis environment. The sample is a packed Windows PE32 executable with the SHA-256 hash

fe1634318e27e3af856506d49a54d1d12e1cf650cbc31eeb0c805949edc8fc85. The sample shows strong behavioral and structural characteristics consistent with the Ploutus-D .NET variant targeting NCR and Diebold hardware.

The analysis was structured in two phases. The static analysis phase examined the binary without executing it, using HashCalc, PE Studio, Detect It Easy, and a hex editor to establish identity, structure, packing status, and embedded artifacts. The dynamic analysis phase involved detonating the packed sample in a controlled Flare VM environment and progressively adapting the emulation environment to coax the malware into revealing its behavior, including dependency hunting, memory inspection, phantom ATM construction, and anti-analysis response observation.

2. Static Analysis

2.1 Cryptographic Hash Identification

HashCalc was used to compute multiple hash values for the sample located at

C:\Malware_Project\samples\Sample

2\fe1634318e27e3af856506d49a54d1d12e1cf650cbc31eeb0c805949edc8fc85.exe.

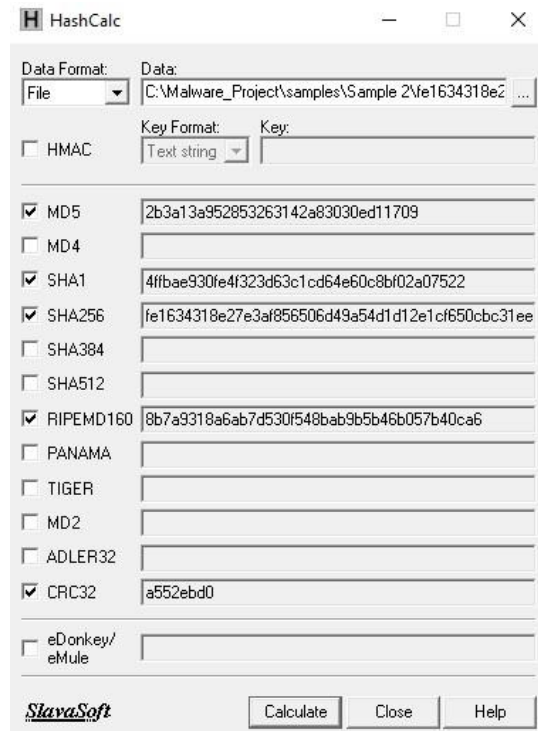


Figure 1: HashCalc showing computed hash values for the Sample 2 executable

The following values were recorded:

MD5: 2b3a13a952853263142a83030ed11709
SHA1: 4ffbae930fe4f323d63c1cd64e60c8bf02a07522
SHA256: fe1634318e27e3af856506d49a54d1d12e1cf650cbc31eeb0c805949edc8fc85
RIPEMD160: 8b7a9318a6ab7d530f548bab9b5b46b057b40ca6
CRC32: a552ebd0

2.2 PE Studio Initial Assessment

PE Studio confirmed the file as a 32-bit Windows GUI executable. The VirusTotal detection ratio is 56 out of 72 engines. The compiler timestamp reads Mon Sep 16 14:53:25 2019. The overall entropy is 6.306 and the file size is 84,480 bytes. No digital signature is present. The import table is not statically enumerated, which is consistent with a packed binary that resolves its imports at runtime.

property	value
md5	86B81D3CE3471B5FF1C1CD5C45014426
sha1	DCECD5367237394B5D582DDA77986C83E6D83B35
sha256	FEAD0633975C6C08F5509A7BD5C34D29BFDCACD3DA47562EFB33121726F77B0
md5-without-overlay	n/a
sha1-without-overlay	n/a
sha256-without-overlay	n/a
first-bytes-hex	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
first-bytes-text	M Z @
file-size	84480 (bytes)
size-without-overlay	n/a
entropy	6.306
imphash	n/a
signature	n/a
entry-point	E8 FD 32 00 00 E9 00 00 00 00 6A 14 68 E0 21 41 00 E8 99 38 00 00 E8 CE 34 00 00 0F B7 F0 6A 02 E8
file-version	n/a
description	n/a
file-type	executable
cpu	32-bit
subsystem	GUI
compiler-stamp	0x5D800455 (Mon Sep 16 14:53:25 2019)
debugger-stamp	n/a
resources-stamp	0x00000000 (empty)
import-stamp	0x00000000 (empty)
exports-stamp	n/a
version-stamp	n/a
certificate-stamp	n/a

File Tree (Left):
 indicators (37)
 virustotal (56/72)
 dos-header (64 bytes)
 dos-stub (176 bytes)
 rich-header (8)
 file-header (Sep.2019)
 optional-header (GUI)
 directories (5)
 sections (98.79%)
 libraries (count)
 imports (count)
 exports (n/a)
 tls-callbacks (n/a)
 resources (manifest) *
 strings (963)
 debug (n/a)
 manifest (asInvoker)
 version (n/a)
 certificate (n/a)
 overlay (n/a)

Status Bar (Bottom):
 sha256: FEAD0633975C6C08F5509A7BD5C34D29BFDCACD3DA47562EFB33121726F77B0 | cpu: 32-bit | file-type: executable | subsystem: GUI | entr

Figure 2: PE Studio output showing file metadata, detection rate, entropy value, and PE header fields

The 37 indicators flagged and the 963 strings in the static table consist largely of garbage sequences from the packed payload rather than meaningful readable text.

2.3 Entropy Analysis

By evaluating the randomness of the file's individual sections, it is evident that the malware utilizes packing to evade detection. The operating system headers and resource sections show normal, readable data levels. However, the primary executable code is entirely encapsulated within a single, highly compressed section. Both the extreme density of this section and its structural naming clearly attribute the obfuscation to the UPX packer.

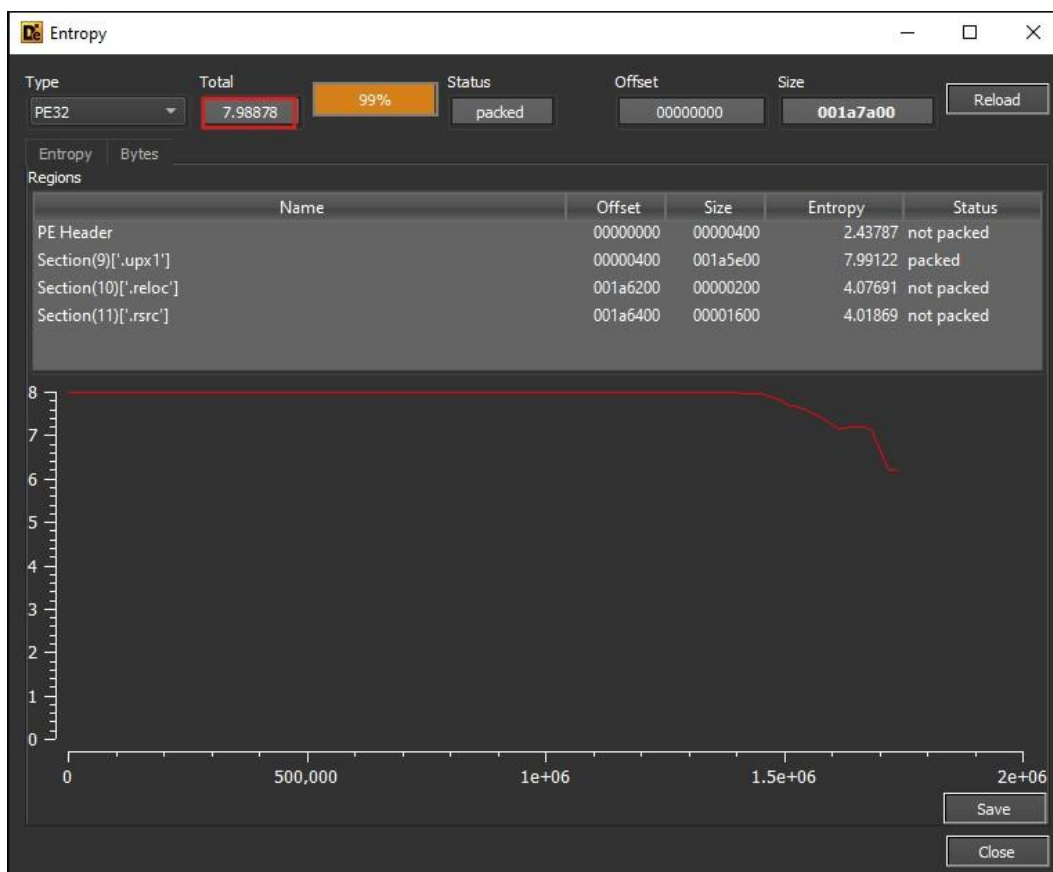


Figure 3: Detect It Easy entropy view showing Section(9) ['.upx1'] with entropy 7.99122, flagged as packed, with overall entropy 7.98878 at 99%

The entropy graph shows an almost perfectly flat line near 8.0 from the start of the file to approximately offset 1,500,000, then a sharp drop where the unpacked .reloc and .rsrc sections begin. This flatness is characteristic of UPX-compressed content where the original executable is compressed into a single dense section.

2.4 Packer and Protector Identification

Detect It Easy confirmed the payload is shielded by VMProtect which adds another layer of protection over the core code. The scanner also identified the linker as Turbo Linker indicating the malware was programmed using Delphi. The structural metadata shows an entry point address of 0x007b6fb9 and a base address of 0x00400000. The internal timestamp reveals a compilation date of July 16 2016. When the executable is fully loaded into memory it spans across 12 distinct sections and occupies an overall image size of 0x003d8000 bytes which equals roughly 3.8 megabytes.

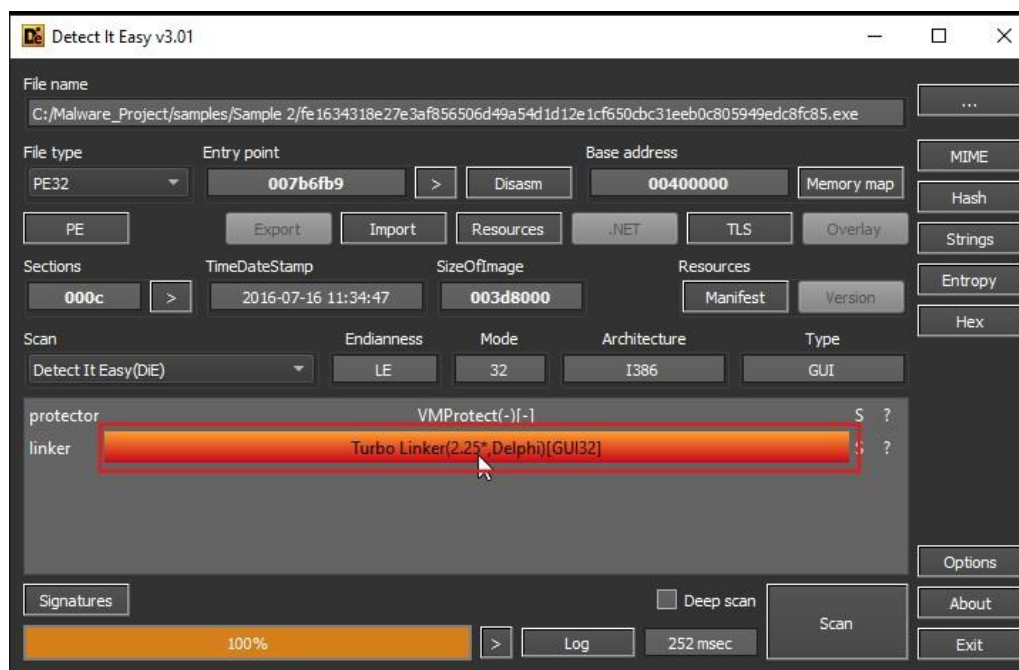


Figure 4: Detect It Easy showing VMProtect protector, Turbo Linker 2.25 Delphi linker, entry point 007b6fb9, compile timestamp July 2016

The identification of Delphi as the underlying language is a critical finding. Ploutus-D, the ATM jackpotting family targeting NCR and Diebold Nixdorf machines, is written in Delphi. VMProtect adds a second protection layer on top of UPX while UPX handles compression, VMProtect virtualizes portions of the code by transforming native x86 instructions into bytecode running on a custom embedded virtual machine, making disassembly significantly harder.

2.5 Hex-Level Inspection with 010 Editor

A manual binary inspection verified the standard Windows executable signature at the beginning of the file. However the subsequent DOS stub region revealed a custom text string replacing the default operating system warning. This specific alteration is a well documented artifact left behind by the UPX packer. Directly following these altered headers the file content abruptly shifts into highly dense and randomized byte sequences which visually confirms the presence of the compressed payload.

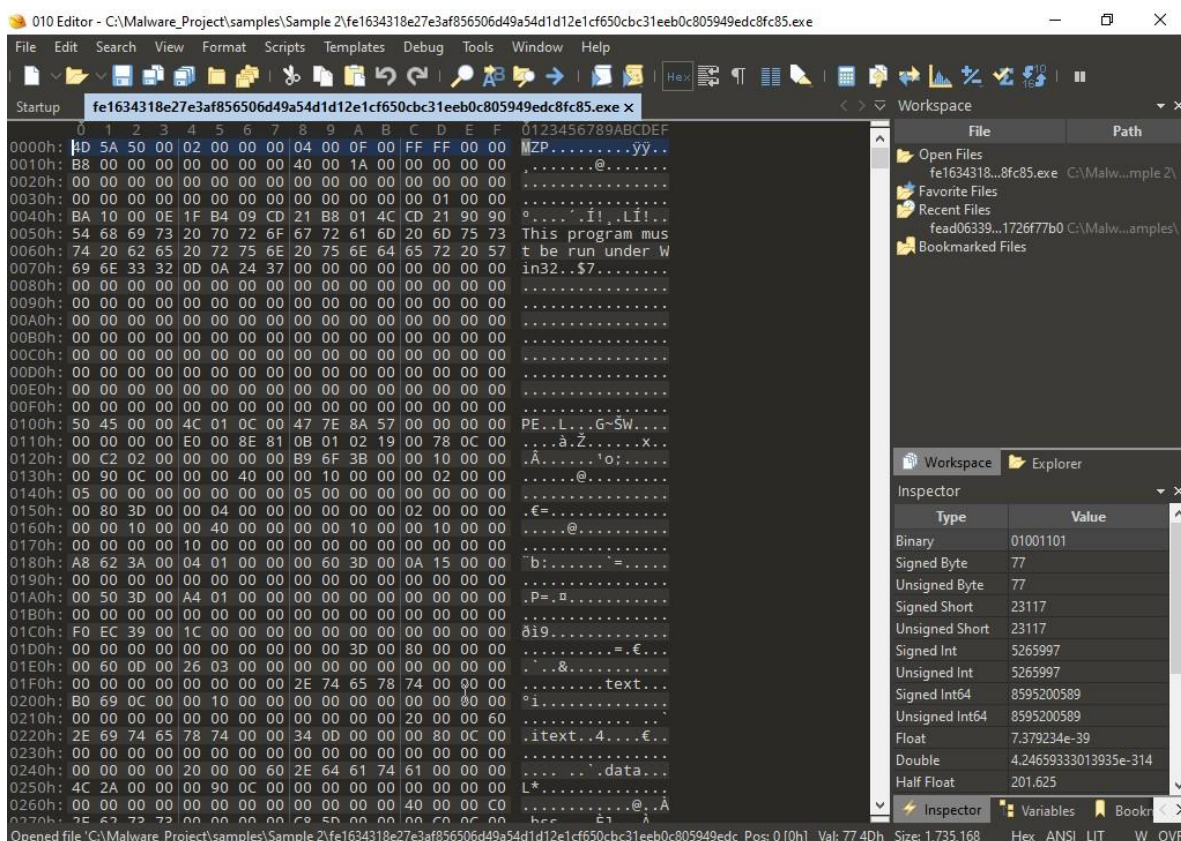


Figure 5: O10 Editor hex view showing the MZ header, UPX-modified DOS stub, and the beginning of highdensity packed content

3. Dynamic Analysis

The sample was executed in a Flare VM environment with administrator privileges. The analysis proceeded through four phases, each building on findings from the previous step.

3.1 Initial Detonation: DLL Dependency Hunting

Process Monitor was deployed during the first detonation with no environmental preparation. The malware process (PID 2152) generated 1,357,219 total events, confirming it entered a sustained execution loop from the start. Filtering for NAME NOT FOUND results showed the malware performing continuous CreateFile operations searching for CSCWCNG.dll. The malware exhaustively searched system paths, including Windows\System32 and various application directories, for CSCWCNG.dll.

Time ...	Process Name	PID	Operation	Path	Result	Detail
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\System32\wow64log.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\msimg32.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\version.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\SysWOW64\edgedgi.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\SysWOW64\vpccs.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\netapi32.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\NETUTILS.DLL	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\wkscdi.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\csapi.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\WindowsCodecs.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\textShaping.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\DWMAPI.DLL	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\SysWOW64\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\System\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Malware_Project\samples\Sample 2\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Program Files (x86)\Common Files\Oracle\Java\javapath_targ...	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Python37\Scripts\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Python37\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Python27\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Python27\Scripts\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\ProgramData\Boxstarter\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\SysWOW64\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\SysWOW64\wbem\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Windows\SysWOW64\WindowsPowerShell\v1.0\CSCWCNG...	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\ProgramData\chocolatey\bin\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Program Files\OpenJDK\openjdk-11.0.12_7\bin\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Program Files\010 Editor\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Program Files\nodejs\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Program Files\Microsoft VS Code\bin\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Users\hushy\AppData\Local\Microsoft\WindowsApps\CSC...	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Tools\Cmder\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\ProgramData\chocolatey\lib\radare2\flare\tools\radare2\bin\...	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Tools\java-deobfuscator-gui\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Tools\Bytecode-Viewer\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Program Files (x86)\Nmap\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\ProgramData\chocolatey\lib\rawcap\tools\rawcap\CSCWCN...	NAME NOT FOUND Desired Access: R...	
9:48:1...	fe1634318e27e...	2152	CreateFile	C:\Tools\lovinstractor\CSCWCNG.dll	NAME NOT FOUND Desired Access: R...	

Figure 6: Process Monitor showing 1,357,219 events with the malware process repeatedly generating NAME NOT FOUND CreateFile operations searching for CSCWCNG.dll across system paths

CSCWCNG.dll is a Windows Cryptographic Next Generation provider DLL. The UPX packer layer in this Ploutus variant requires this DLL as a cryptographic dependency to decrypt the inner VMProtect-protected payload. Because the analysis VM lacked this specific legacy cryptographic provider, the packer could not unlock the encrypted core and looped endlessly through the system PATH. This behavior functions as a passive anti-analysis gate: the malware produces no actionable artifacts until the dependency is satisfied.

3.2 Live Memory Extraction via Process Hacker

To determine whether any portion of the payload had decrypted despite the missing dependency, the running process was inspected directly using Process Hacker. A full string search of the process memory space returned 3,494 results. All were obfuscated or encoded fragments from the still-encrypted inner payload rather than readable text.

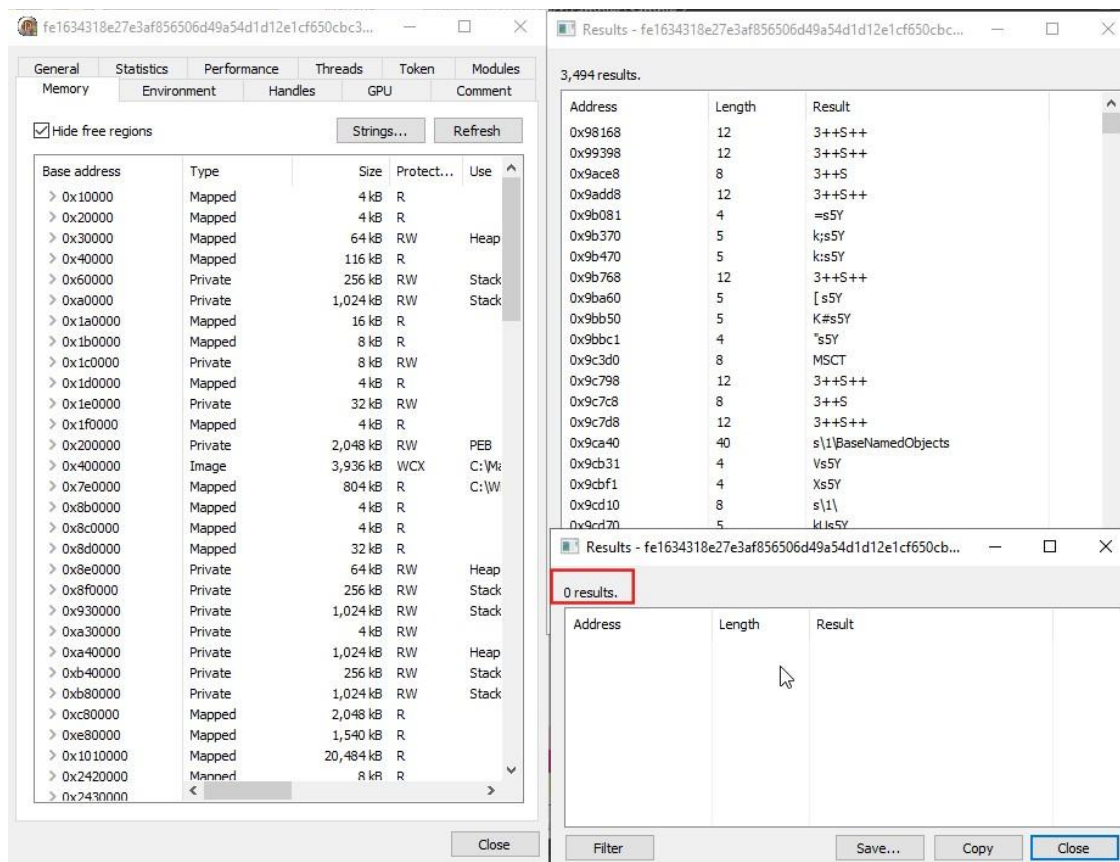


Figure 7: Process Hacker memory string search returning 3,494 obfuscated results and a second targeted search for ATM vendor terms returning 0 results

A second targeted search for ATM vendor-specific identifiers including CASHDISPENSER3Lib, Agilis, and msxfs returned 0 results. This confirmed that despite the extensive loop activity, the core jackpotting payload remained fully encrypted in memory. The UPX decompression stage could not proceed without CSCWCNG.dll, so the VMProtect layer and the ATM payload beneath it were never reached.

3.3 Phantom ATM Construction: Dummy File Emulation

Based on the Procmon findings and the identification of the sample as a .NET-based Ploutus variant, .NET Framework 3.5 was installed on the VM and zero-byte dummy files matching the COM Interop wrapper names that Ploutus targets were created and placed alongside the malware executable in the sample directory.

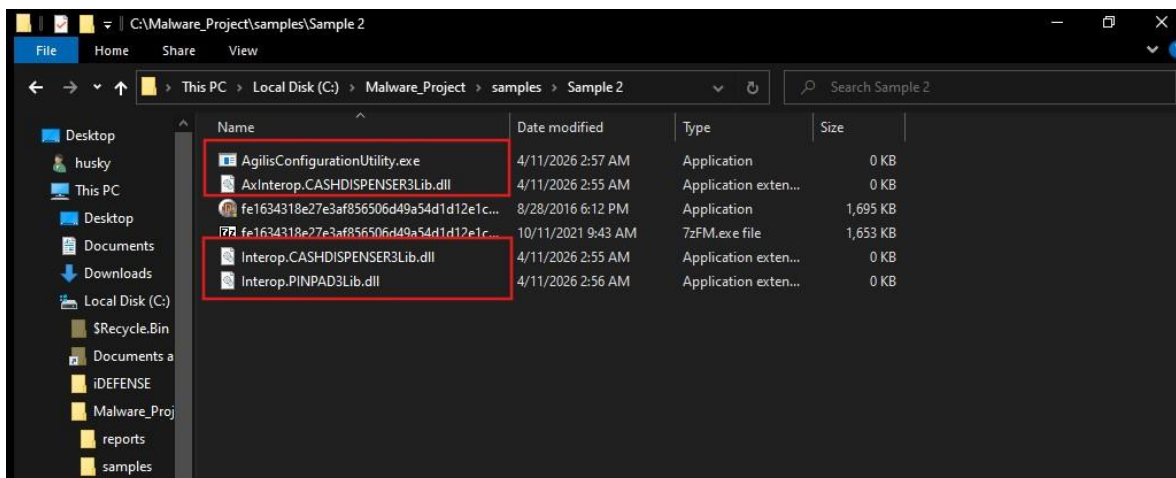


Figure 8: Sample 2 directory showing the dummy files AgilisConfigurationUtility.exe, AxInterop.CASHDISPENSER3Lib.dll, Interop.CASHDISPENSER3Lib.dll, and Interop.PINPAD3Lib.dll created alongside the malware executable

The four dummy files created were AgilisConfigurationUtility.exe, AxInterop.CASHDISPENSER3Lib.dll, Interop.CASHDISPENSER3Lib.dll, and Interop.PINPAD3Lib.dll. AgilisConfigurationUtility.exe is associated with Diebold Nixdorf ATM management software. The CASHDISPENSER3Lib interop assemblies are .NET COM wrappers for the NCR cash dispenser hardware interface, and Interop.PINPAD3Lib.dll is the equivalent wrapper for the PIN pad. Ploutus-D uses .NET File.Exists() checks to verify these files are present before proceeding. Dropping zero-byte stubs with the correct names satisfied those checks and convinced the malware it was executing on a legitimate ATM workstation.

3.4 Anti-Analysis and Tamper Detection Response

With the phantom ATM environment in place, API Monitor v2 (32-bit) was configured with hooks targeting .NET Framework assembly loading and reflection calls, and the sample was re-executed. This session captured 10,251 calls, which is a significant increase from the previous dormant run and confirmed the malware had moved past the dependency gate.

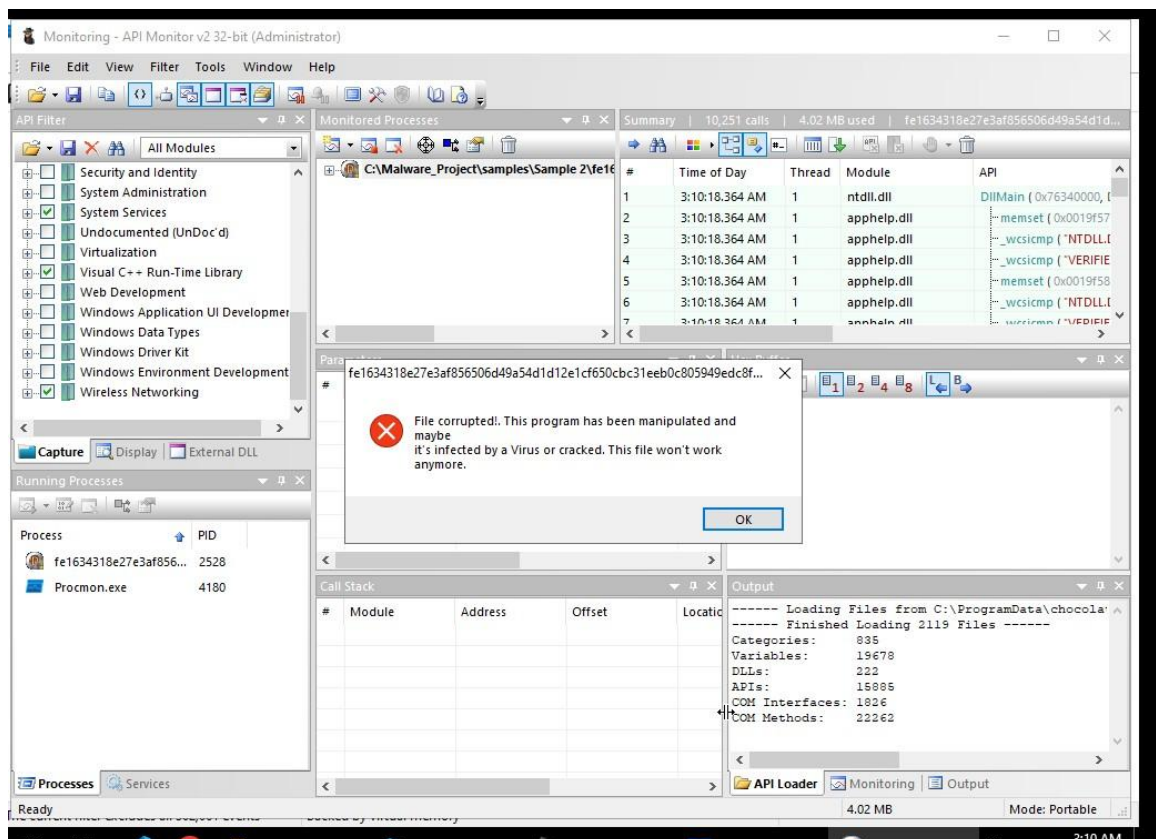


Figure 9: API Monitor showing the malware displaying a fake decoy dialog reading "File corrupted!. This program has been manipulated and maybe it's infected by a Virus or cracked. This file won't work anymore."

The malware woke up, detected the dummy files, and began initialization. However, the packer then performed a self-integrity check, calculated a checksum of its own loaded code in memory, and detected the foreign hooks injected by API Monitor. Upon detecting the discrepancy, it triggered a suicide switch and displayed the dialog: "File corrupted!. This program has been manipulated and maybe it's infected by a Virus or cracked. This file won't work anymore." This message is not a genuine Windows error. It is a deliberate decoy designed to make the analyst believe the file is non-functional and abandon the investigation. The malware intentionally terminated execution before the cash dispenser commands could be reached or captured.

4. Conclusion

The analyzed executable is a Delphi-compiled ATM jackpotting binary consistent with the Ploutus-D family, compiled in July 2016, packed with UPX, and further protected with VMProtect. The SHA-256 hash is `fe1634318e27e3af856506d49a54d1d12e1cf650cbc31eeb0c805949edc8fc85`. The overall entropy of 7.98878 at 99% packing confidence and the explicit `.upx1` section name leave no ambiguity about the protection layer.

Dynamic analysis revealed a four-stage defensive architecture. The initial detonation exposed a `CSCWCNG.dll` dependency gate that traps the packer in a PATH-exhaustion loop, generating over 1.3 million file system events while the payload stays encrypted. Memory inspection confirmed the inner payload remained fully ciphertext in RAM with zero vendor-specific strings decrypted. Phantom ATM construction using zero-byte COM interop stubs successfully bypassed the `File.Exists()` environmental checks, causing the malware to advance past the dependency loop. At that point, the self-integrity check detected the API Monitor hooks and the suicide switch fired, displaying a fake corruption error to deflect further analysis.

To reach the cash dispenser commands, the `CSCWCNG.dll` check and the integrity check must both be neutralized. The recommended path is manual unpacking in `x32dbg`: locate and patch the cryptographic dependency check, reach the OEP after UPX decompression, dump the decompressed image, and address the VMProtect virtualization layer to recover the original Delphi logic. The combination of UPX, VMProtect, cryptographic dependency gating, vendor-specific environmental checks, and memory-integrity suicide switching represents one of the most layered anti-analysis profiles documented in the ATM malware category.